

Predicting Student Test Scores

RHOUDDAL SAAD
SEK SOPHEAK VOATEI
THONG OUSAPHEA
VAITHILINGAM VITHUSON

19 mars 2026

Sommaire

- 1 Introduction
- 2 Analyse des données
- 3 Prétraitement des données
- 4 Modélisation
- 5 Résultats

Contexte

Compétition Kaggle

Playground Series – S6E1 : Regression with an Academic Success Dataset

Objectif

Prédire le score final d'un étudiant à un examen en fonction de ses caractéristiques comportementales, académiques et personnelles.

Présentation du Dataset

Présentation de train.csv

630 000 observations – 12 variables explicatives + 1 cible

#	Variable	Type
0	id	int
1	age	int
2	gender	str
3	course	str
4	study_hours	float
5	class_attendance	float
6	internet_access	str

#	Variable	Type
7	sleep_hours	float
8	sleep_quality	str
9	study_method	str
10	facility_rating	str
11	exam_difficulty	str
12	exam_score (cible)	float

Numériques

4 variables

Catégorielles

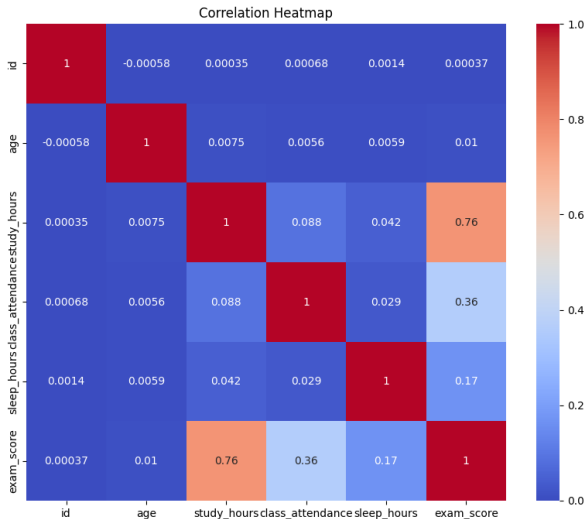
7 variables

Statistiques descriptives

Variable	Moyenne	Écart-type	Min	Médiane	Max
Âge	20.55	2.26	17.0	21.0	24.0
Heures d'étude	4.00	2.36	0.08	4.00	7.91
Assiduité (%)	71.99	17.43	40.6	72.6	99.4
Heures de sommeil	7.07	1.74	4.10	7.10	9.90
Score (cible)	62.51	18.92	19.6	62.6	100.0

Table – Statistiques descriptives (N = 630 000)

Matrice des corrélations



Nettoyage et sélection initiale

Suppression de la colonne id

L'identifiant unique de chaque étudiant n'a aucune valeur prédictive.

Séparation des données

- **X** : matrice des caractéristiques (variables explicatives)
- **y** : vecteur cible (`exam_score`)

Split train / validation

Division en **80% entraînement** / **20% validation** locale pour évaluer les modèles avant soumission Kaggle.

Transformation des variables

Construction d'un **ColumnTransformer** :

Variables numériques → **StandardScaler**

Centre et réduit (moyenne = 0, écart-type = 1).

Variables : age, study_hours, class_attendance, sleep_hours

Variables catégorielles ordinales → **OrdinalEncoder**

Conserve l'ordre logique (ex : low < medium < high).

Variables : facility_rating, sleep_quality, exam_difficulty

Variables catégorielles nominales → **OneHotEncoder**

Crée des colonnes binaires.

Variables : gender, course, study_method

Méthodologie de validation

Validation Croisée K-Fold ($K = 5$)

- ① Le jeu d'entraînement est divisé en 5 plis égaux
- ② Le modèle s'entraîne sur $K - 1 = 4$ plis, est évalué sur le 5^e
- ③ L'opération est répétée 5 fois : chaque pli sert une fois de test
- ④ Performance retenue : **moyenne** des 5 scores RMSE

Régression Linéaire

Principe

Modéliser `exam_score` comme une somme pondérée des caractéristiques :

$$\min_{\beta} \left(\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right)$$

Sélection de variables via `SelectKBest (f_regression)`

Valeurs testées : $k \in \{\text{all}, 10, 8\}$

- Meilleur paramètre retenu : `k = 'all'` (toutes les variables)
- La régression linéaire tire profit de l'ensemble des caractéristiques

Résultat

RMSE = 8.887 $R^2 = 0.778$ — Relation linéaire très forte

Support Vector Regression

Principe

Cherche une droite entourée d'un **tube de tolérance** ϵ : les points à l'intérieur sont ignorés, seuls ceux à l'extérieur (les *vecteurs de support*) influencent le modèle.

Hyperparamètre de régularisation

- C élevé $\rightarrow$ pénalité forte $\rightarrow$ risque de surapprentissage
- C faible $\rightarrow$ modèle plus robuste et généralisable

Résultat

Meilleur paramètre : $C = 1.0$ $RMSE = 8.892$ $R^2 = 0.778$

Arbre de Décision

Principe

Segmente l'espace des données par des règles successives. À chaque nœud : minimiser la variance (MSE) dans les groupes créés.

Hyperparamètres

- `max_depth` : profondeur maximale de l'arbre (nombre de règles successives)
- `min_samples_split` : taille minimale d'un nœud pour pouvoir le diviser

Résultat

`k = 10, max_depth = 10, min_samples_split = 10`
RMSE = 9.637 $R^2 = 0.739$

Méthodes Ensemblistes

Principe

Entraîner plusieurs arbres **indépendants** sur des sous-échantillons aléatoires et agréger leurs prédictions par moyenne.

Random Forest – `n_estimators = 30`, `max_depth = 20`

Bootstrap + sous-ensemble de variables aléatoire à chaque nœud.

RMSE = **9.200** $R^2 = 0.762$

Extra Trees – `n_estimators = 100`, `max_depth = 20`

Seuils de coupure **entièrement aléatoires** → variance encore plus réduite.

RMSE = **9.083** $R^2 = 0.768$

Boosting – CatBoost

Principe du Boosting

Arbres construits **séquentiellement** : chaque arbre corrige les erreurs du précédent. L'algorithme se concentre progressivement sur les cas difficiles à prédire.

CatBoost – Arbres symétriques

Hyperparamètres optimisés :

- `iterations` = 200 : nombre total d'arbres séquentiels
- `depth` = 8 : profondeur des arbres symétriques
- `learning_rate` = 0.1 : agressivité de correction à chaque étape

Résultat

RMSE = 8.800 $R^2 = 0.782$

Boosting – LightGBM

Principe

Au lieu de croître niveau par niveau, LightGBM développe ses arbres **feuille par feuille** : à chaque étape, il choisit la feuille qui réduit le plus l'erreur globale.

Hyperparamètres optimisés

- `n_estimators` = 400 : nombre d'arbres construits
- `learning_rate` = 0.1 : taux d'apprentissage
- `num_leaves` = 63 : capture des interactions subtiles
- `subsample` = 0.8 : sous-échantillonnage pour éviter le surapprentissage

Résultat

RMSE = 8.770 R^2 = 0.784 — Meilleur modèle individuel

Stacking

Principe

Un **méta-modèle** apprend à pondérer les prédictions de chaque modèle de base pour maximiser la précision globale.

Méta-modèle : RidgeCV

Modèle linéaire simple et régularisé : évite la mémorisation des erreurs des modèles de base tout en ajustant automatiquement les poids optimaux.

Résultat

RMSE = 8.504 $R^2 = 0.797$

Comparaison des modèles

Modèle	MAE	RMSE	R ²
Stacking_Total	6.694	8.388	0.8022
LightGBM	6.984	8.762	0.7841
CatBoost	7.021	8.795	0.7825
LinearRegression	7.021	8.887	0.7779
SVM	7.093	8.892	0.7777
ExtraTrees	7.093	9.083	0.7680
RandomForest	7.327	9.200	0.7620
DecisionTree	7.712	9.637	0.7389

Table – Performances sur le jeu de validation local (20%)

Interprétation des résultats

- Le Stacking est le meilleur modèle ($RMSE = 8.504$)
- Relation entre les variables prédictives et la note linéaire.
- Boosting est supérieur au Bagging

Comparaison : Prédiction/Réalité

Comparaison : Réalité vs Prédictions

